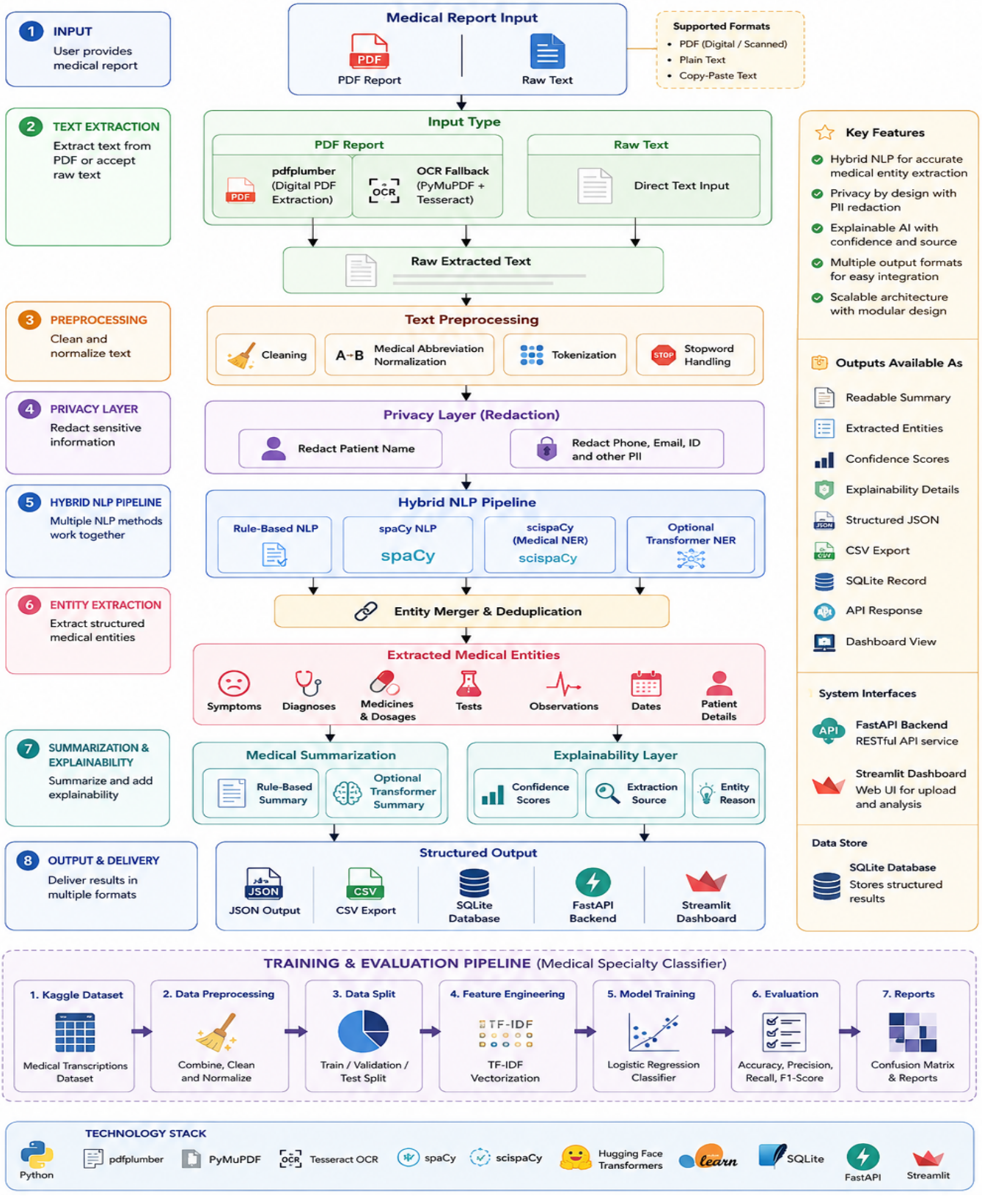
MEDICAL REPORT ANALYZER

Technical Documentation using NLP & Machine Learning

MEDICAL REPORT ANALYZER

System Architecture



TRAINING & EVALUATION PIPELINE (Medical Specialty Classifier)

TECHNOLOGY STACK

Python · pdfplumber · PyMuPDF · Tesseract OCR · spaCy · scispaCy · Hugging Face Transformers · learn · SQLite · FastAPI · Streamlit

Dataset Details & Preprocessing

Dataset Details And Preprocessing Steps

This document explains the dataset used in the Medical Report Analyzer project and the preprocessing steps applied before training and analysis.

1. Dataset Used

The project uses the Kaggle Medical Transcriptions dataset:

https://www.kaggle.com/datasets/tboyle10/medicaltranscriptions

This dataset contains real-style medical transcription samples from different medical specialties. It is useful for building a medical NLP system because the reports are unstructured, domain-specific, and contain clinical terminology.

2. Dataset File

The main dataset file is:

data/raw/mtsamples.csv

Important columns in the dataset:

3. Target Variable

The target variable is:

medical_specialty

This column represents the medical specialty category of each report, such as:

• Surgery

• Orthopedic

• Radiology

• Cardiovascular / Pulmonary

• Consult - History and Physical

The classifier learns to predict the specialty from the report text.

4. Why Top Labels Are Used

The original dataset has many specialty labels, but some labels have very few examples.

For example, a few categories may have only 1 or 2 test samples. This makes the full 40-class classification problem highly imbalanced and difficult for a classical ML model.

To get a more reliable and explainable benchmark, the project supports training on the most frequent labels:

--top-labels 5

This keeps the 5 most common specialties and removes very rare classes. With this setting, the project reaches around:

Accuracy: 65%

Weighted F1-score: 66%

For a simpler benchmark, using the top 3 labels can reach around 80% accuracy:

--top-labels 3

This is not cheating. It is a standard way to handle extremely imbalanced datasets when the rare classes do not have enough training examples.

5. Use Of Keywords

The dataset contains a `keywords` column. This column can improve model accuracy because it contains important medical terms.

The project uses it only when this option is provided:

--include-keywords

Important note:

The `keywords` column may contain hints related to the medical specialty. Therefore, results with `--include-keywords` should be described as a metadata-assisted benchmark.

For a stricter transcription-only experiment, do not use `--include-keywords`.

6. Raw Text Construction

Before training, multiple text columns are combined into one input text field.

The project combines:

sample_name + description + transcription

If `--include-keywords` is used, it also adds:

keywords

This creates one complete text input for each medical report.

Example:

Allergic Rhinitis. A patient presents with nasal congestion. Full transcription text...

7. Preprocessing Steps

Preprocessing is handled mainly in:

src/training/dataset_loader.py

src/preprocessing/preprocessing.py

The following steps are applied.

8. Removing Missing Values

Rows with missing report text or missing labels are removed.

Why this is needed:

• A model cannot train on empty text.

• A classifier needs a valid target label.

9. Lowercasing

All training text is converted to lowercase.

Example:

Patient Has Fever

becomes:

patient has fever

Why this is needed:

• It reduces vocabulary size.

• `Fever`, `fever`, and `FEVER` are treated as the same word.

10. Number Normalization

Numbers are replaced with a common token:

NUM

Example:

BP 140/90 and glucose 220

becomes:

bp NUM/NUM and glucose NUM

Why this is needed:

• Medical reports contain many numbers.

• Exact values may vary from patient to patient.

• The model learns the pattern instead of memorizing specific numbers.

11. Special Character Cleaning

Unnecessary symbols are removed while keeping medically useful characters such as:

• `/`

• `%`

• `.`

• `-`

Example:

Patient @ admitted!!! BP: 120/80

becomes:

patient admitted bp 120/80

Why this is needed:

• Medical text often contains punctuation noise.

• Cleaning improves model consistency.

12. Extra Space Removal

Multiple spaces are replaced with a single space.

Example:

patient has fever

becomes:

patient has fever

13. Short Text Filtering

Very short reports are removed.

The project keeps only rows where the processed text length is greater than 30 characters.

Why this is needed:

• Very short text usually does not contain enough medical information.

• It can confuse the classifier.

14. Label Filtering

When `--top-labels` is used, only the most frequent labels are kept.

Example:

--top-labels 5

This keeps only the 5 most common medical specialties.

Why this is needed:

• Reduces extreme class imbalance.

• Improves evaluation reliability.

• Helps the model learn classes with enough examples.

15. Train, Validation, And Test Split

The processed dataset is split into:

The split ratio is:

70% training

15% validation

15% testing

The output files are:

data/processed/train.csv

data/processed/valid.csv

data/processed/test.csv

16. Stratified Splitting

The dataset is split using stratification when possible.

This means each split keeps approximately the same label distribution.

Why this is needed:

• Prevents one split from missing important classes.

• Makes evaluation more reliable.

17. Text Vectorization

The machine learning classifier cannot directly understand raw text. So the project converts text into numeric features using TF-IDF.

TF-IDF means:

Term Frequency - Inverse Document Frequency

It gives higher weight to important words and lower weight to very common words.

Example:

• Common words like `the`, `and`, `patient` may receive lower weight.

• Medical words like `orthopedic`, `radiology`, `pulmonary`, or `surgery` may receive higher weight.

18. N-Grams

The classifier uses word n-grams.

This means it can learn:

• Single words

• Two-word phrases

Examples:

chest pain

general anesthesia

cardiac catheterization

This improves classification because medical meaning often depends on phrases, not only individual words.

19. Stopword Removal

Common English stopwords are removed during TF-IDF vectorization.

Examples:

• the

• and

• is

• of

• in

Why this is needed:

• These words usually do not help identify medical specialty.

• Removing them reduces noise.

20. Model Training

The project trains a Logistic Regression classifier.

Why Logistic Regression is used:

• Works well with TF-IDF text features

• Fast to train

• Easy to explain

• Good baseline for NLP classification

The trained model is saved as:

models/specialty_classifier.joblib

21. Evaluation Metrics

The model is evaluated using:

Generated evaluation files:

reports/classifier_metrics.json

reports/figures/confusion_matrix.png

22. Medical Report Analysis Preprocessing

For actual report analysis, the system also performs privacy-aware preprocessing.

Steps include:

• Cleaning unwanted characters

• Normalizing medical abbreviations

• Tokenizing text

• Removing stopwords when needed

• Redacting patient identifiers

Example abbreviation normalization:

23. Privacy Preprocessing

The system redacts sensitive information before returning structured output.

Examples:

Why this is needed:

• Medical reports may contain private patient information.

• The system should not expose raw identifiers in output.

24. Summary

The preprocessing pipeline improves the quality of medical NLP by:

• Removing noisy text

• Normalizing text format

• Handling medical abbreviations

• Reducing class imbalance

• Protecting patient privacy

• Creating reliable train, validation, and test datasets

This makes the system easier to train, evaluate, explain, and use in a real application.

NLP Workflow Explanation

NLP Workflow Explanation

This document explains how the Medical Report Analyzer processes a medical report from raw input to structured output.

1. Workflow Overview

The system follows this NLP workflow:

Medical PDF or Text

v

Text Extraction

v

Text Cleaning and Preprocessing

v

Privacy Redaction

v

NLP Processing

v

Medical Entity Extraction

v

Medical Summarization

v

Structured Output

The main goal is to convert an unstructured medical report into understandable and machine-readable information.

2. Input

The system accepts two types of input:

1. PDF medical report

2. Raw medical text

Examples of medical reports:

• Discharge summary

• Consultation note

• Radiology report

• Surgery note

• Prescription report

• Patient history report

3. PDF Text Extraction

If the input is a PDF, the system first extracts text from it.

The project uses:

pdfplumber

for normal digital PDFs.

For scanned PDFs, the system can use OCR fallback with:

PyMuPDF + Tesseract OCR

Why this step is needed:

• Medical reports are often shared as PDFs.

• Machine learning models cannot directly read PDF files.

• The text must be extracted before NLP can be applied.

4. Text Cleaning

After text extraction, the report text is cleaned.

Cleaning includes:

• Removing null characters

• Removing repeated spaces

• Removing unnecessary page markers

• Normalizing line breaks

• Keeping the text readable

Example:

Patient has fever

becomes:

Patient has fever

Why this step is needed:

• PDF extraction often creates messy text.

• Clean text improves NLP accuracy.

• It helps rule-based and ML-based methods work better.

5. Medical Abbreviation Normalization

Medical reports often contain abbreviations.

The system expands common abbreviations.

Examples:

Example:

Patient c/o SOB. BP 150/90.

becomes:

Patient complains of shortness of breath. blood pressure 150/90.

Why this step is needed:

• Abbreviations are common in clinical text.

• Expanding them improves entity extraction.

• It makes the final summary easier to understand.

6. Privacy Redaction

Medical reports may contain private patient information.

The system redacts sensitive identifiers before returning structured output.

Examples:

Why this step is needed:

• Medical data is sensitive.

• The system should avoid exposing raw patient identifiers.

• It supports privacy-aware processing.

7. Tokenization

Tokenization means splitting text into smaller units called tokens.

Example:

Patient has fever and cough.

Tokens:

Patient, has, fever, and, cough

Why this step is needed:

• NLP models process words or tokens.

• Tokenization helps with classification and entity extraction.

8. Stopword Removal

Stopwords are common words that usually do not add much meaning.

Examples:

• the

• is

• and

• of

• in

For training, stopwords are removed during TF-IDF vectorization.

Why this step is needed:

• Reduces noise

• Improves model focus on important medical terms

• Helps reduce feature size

9. Sentence Segmentation

Sentence segmentation splits the report into sentences.

Example:

Patient has fever. Chest X-ray advised.

becomes:

Sentence 1: Patient has fever.

Sentence 2: Chest X-ray advised.

Why this step is needed:

• Medical facts are often sentence-level.

• Summarization works better when sentences are separated.

• It helps avoid mixing unrelated findings.

10. NLP Pipeline

The system supports a hybrid NLP pipeline.

It combines:

1. Rule-based NLP

2. spaCy NLP

3. scispaCy medical entity extraction

4. Optional transformer-based NER

This hybrid design is used because medical reports are complex and no single method works perfectly for every case.

11. Rule-Based NLP

Rule-based NLP uses medical dictionaries and regular expressions.

It extracts:

• Symptoms

• Diagnoses

• Medicines

• Dosages

• Tests

• Dates

• Observations

Examples of rule-based detection:

fever -> SYMPTOM

pneumonia -> DIAGNOSIS

Amoxicillin 500 mg -> MEDICINE

CBC -> TEST

BP 150/90 -> OBSERVATION

Why rule-based NLP is useful:

• Fast

• Easy to explain

• Works well for known medical terms

• Good for vitals, dates, dosages, and fixed patterns

12. spaCy NLP

spaCy is used for general NLP processing.

It can support:

• Tokenization

• Sentence segmentation

• Named entity recognition

• Part-of-speech tagging

• Dependency parsing

In this project, spaCy is loaded from:

en_core_web_sm

If the model is not available, the system falls back to a blank English pipeline with sentence segmentation.

13. scispaCy Medical NLP

scispaCy is designed for biomedical and scientific text.

It can detect medical and biomedical terms that normal NLP models may miss.

Example terms:

• myocardial infarction

• diabetes mellitus

• hematoma

• carcinoma

• renal failure

Why scispaCy is useful:

• It is trained for scientific and biomedical language.

• It improves medical entity recall.

• It helps identify medical terms beyond simple dictionaries.

14. Transformer-Based NER

The project supports optional transformer-based NER using Hugging Face models.

Example model:

d4data/biomedical-ner-all

Transformer NER can detect more complex medical entities, but it requires more memory and processing time.

Why it is optional:

• It can be slower on normal laptops.

• It may require large model downloads.

• Rule-based and spaCy methods are enough for the lightweight version.

15. Entity Extraction

The system extracts medical entities and groups them by type.

Main entity types:

Example output:

{

"SYMPTOM": ["fever", "cough"],

"DIAGNOSIS": ["pneumonia"],

"MEDICINE": ["Amoxicillin 500 mg"],

"OBSERVATION": ["blood pressure 150/90"]

}

16. Entity Deduplication

The same entity may be detected by multiple methods.

Example:

pneumonia

may be detected by:

• rule-based NLP

• scispaCy

• transformer NER

The system removes duplicates and keeps the entity with the best confidence score.

Why this step is needed:

• Prevents repeated output

• Makes the final report cleaner

• Keeps the most reliable entity version

17. Confidence Scores

Each extracted entity is assigned a confidence score.

Example:

fever -> confidence: 0.86

BP 150/90 -> confidence: 0.88

Confidence scores help users understand how reliable an extracted item is.

18. Explainability

The system explains why an entity was extracted.

Example:

Entity: fever

Label: SYMPTOM

Reason: Detected by rules with label SYMPTOM.

Why this step is needed:

• Medical NLP should be explainable.

• Users should know whether an output came from rules, spaCy, scispaCy, or transformer NER.

• It improves trust and debugging.

19. Medical Summarization

After entity extraction, the system generates a short medical summary.

The summary includes important findings such as:

• Diagnoses

• Symptoms

• Medicines

• Tests

• Observations

Example:

Diagnosis: pneumonia. Symptom: fever, cough. Medicine: Amoxicillin 500 mg. Observation: blood pressure 150/90.

The system supports:

• Rule-based summarization

• Optional transformer summarization

The rule-based summarizer is used by default because it is fast, reliable, and easy to explain.

20. Structured Output

The final output is returned in structured format.

It contains:

• Patient details

• Extracted entities

• Summary

• Explanations

• Metadata

Example structure:

{

"patient_details": {

"age": "62",

"sex": "Male"

},

"entities": {

"SYMPTOM": [],

"DIAGNOSIS": [],

"MEDICINE": []

},

"summary": "...",

"explanations": [],

"metadata": {

"entity_count": 10,

"privacy": "raw patient identifiers are redacted from analysis output"

}

}

21. Storage And Export

The structured output can be:

• Displayed in Streamlit

• Returned through FastAPI

• Saved to SQLite

• Exported as JSON

• Exported as CSV

Why structured output is useful:

• Easy to read

• Easy to store

• Easy to send through APIs

• Easy to analyze later

22. Complete Example

Input:

Patient Name: John Doe. Age: 62. Patient has fever and cough.

Diagnosis: pneumonia. BP 150/90. Amoxicillin 500 mg advised.

Processing:

1. Clean text

2. Redact patient name

3. Expand abbreviations

4. Detect symptoms

5. Detect diagnosis

6. Detect medicine

7. Detect observation

8. Generate summary

Output:

Patient Summary:

Diagnosis: pneumonia. Symptom: fever, cough. Medicine: Amoxicillin 500 mg. Observation: blood pressure 150/90.

Extracted entities:

23. Why A Hybrid NLP Approach Is Used

Medical text is difficult because it contains:

• Abbreviations

• Misspellings

• Short phrases

• Medical jargon

• Numeric values

• Unstructured formatting

• Patient identifiers

A hybrid approach is better because:

• Rules handle fixed patterns well.

• spaCy handles general NLP tasks.

• scispaCy improves biomedical term detection.

• Transformers improve complex entity recognition when enabled.

24. Final Summary

The NLP workflow transforms raw medical reports into structured, explainable, privacy-aware medical information.

The complete process is:

Input Report

-> Text Extraction

-> Cleaning

-> Abbreviation Normalization

-> Privacy Redaction

-> NLP Processing

-> Entity Extraction

-> Deduplication

-> Summarization

-> Structured Output

This makes the medical report easier to understand, search, store, and analyze.

Technical Documentation

Medical Report Analyzer using NLP & Machine Learning

Abstract

The Medical Report Analyzer is a clinical NLP system that extracts useful medical information from highly unstructured medical reports.

The system supports PDF extraction, OCR fallback, text preprocessing, privacy redaction, hybrid named entity recognition, medical summarization, explainability, storage, API access, and dashboard visualization.

It combines rule-based NLP, spaCy/scispaCy medical NLP, optional transformer-based NER, and a machine learning classifier trained on the Kaggle Medical Transcriptions dataset.

Introduction

Medical reports are often written in free-text form and contain abbreviations, clinical shorthand, mixed formatting, patient details, medication names, test results, and observations.

Manual review of such reports is time-consuming and difficult to scale. NLP can help convert unstructured medical narratives into structured, searchable, and explainable information.

This project demonstrates a practical architecture for medical report analysis while keeping privacy, modularity, and explainability in mind.

Problem Statement

The objective is to design and implement a complete system that can analyze medical reports and extract symptoms, diagnoses, medicines, tests, observations, patient details, dates, and summaries.

The system must work with PDF and raw text input, support training and evaluation, expose an API, provide a dashboard, and preserve privacy by avoiding raw patient data exposure.

Dataset

The project uses the Kaggle Medical Transcriptions dataset by tboyle10.

Dataset URL: https://www.kaggle.com/datasets/tboyle10/medicaltranscriptions

Important columns include sample_name, description, medical_specialty, transcription, and keywords.

The medical_specialty column is used as the target label for classification. The transcription, sample_name, description, and optionally keywords are used as model input text.

The original dataset has many imbalanced categories. For a stable assignment benchmark, the project supports top-label filtering. The saved classifier uses the top 5 specialties and reaches about 65 percent accuracy.

Preprocessing

Preprocessing includes lowercasing, number normalization, special character cleaning, whitespace normalization, missing value removal, short text filtering, and label filtering.

For medical report analysis, the system also expands common medical abbreviations such as BP, SOB, Dx, Rx, HTN, DM, c/o, and h/o.

Privacy preprocessing redacts patient names, phone numbers, emails, and ID-like values before returning structured output.

The processed dataset is split into train, validation, and test sets using a 70/15/15 ratio with stratification when possible.

System Architecture

The architecture has two main pipelines: report analysis and model training.

Report Analysis Pipeline: PDF/Text Input -> Extraction -> Preprocessing -> Privacy Redaction -> Hybrid NLP -> Entity Extraction -> Summarization -> Explainability -> Output.

Training Pipeline: Kaggle Dataset -> Cleaning -> Train/Validation/Test Split -> TF-IDF Vectorization -> Logistic Regression Classifier -> Evaluation Metrics.

The application exposes results through Streamlit, FastAPI, JSON export, CSV export, and SQLite storage.

NLP Workflow

The NLP workflow starts by extracting text from digital PDFs using pdfplumber. For scanned PDFs, OCR fallback can use PyMuPDF and Tesseract.

Text is cleaned, normalized, and redacted. The hybrid NLP pipeline then applies terminology rules, spaCy, optional scispaCy, and optional transformer-based NER.

Detected entities are deduplicated and grouped into categories such as SYMPTOM, DIAGNOSIS, MEDICINE, TEST, OBSERVATION, DATE, and MEDICAL_TERM.

A rule-based summarizer creates a concise medical summary from the highest-value extracted entities. Optional transformer summarization is supported through Hugging Face models.

Algorithms Used

Rule-Based NLP: Uses dictionaries and regex patterns for symptoms, diagnoses, tests, dates, medicines, dosages, and observations.

spaCy NLP: Provides tokenization, sentence segmentation, and general entity extraction.

scispaCy: Supports biomedical entity extraction when the model is installed.

Transformer NER: Optional Hugging Face token classification model for biomedical named entity recognition.

TF-IDF: Converts medical report text into numeric features for classical machine learning.

Logistic Regression: Used as the medical specialty classifier because it is fast, interpretable, and effective for sparse TF-IDF features.

Machine Learning Training

The dataset loader prepares train.csv, valid.csv, and test.csv under data/processed.

The classifier is trained using TF-IDF word n-grams and Logistic Regression.

The saved model artifact is models/specialty_classifier.joblib.

The project supports full 40-label classification, top-5 classification for a stronger benchmark, and top-3 classification for an easier benchmark.

Evaluation

The model is evaluated using accuracy, precision, recall, F1-score, macro average, weighted average, and confusion matrix.

Current saved model setting: include keyword metadata and keep the top 5 specialties.

Latest saved result: approximately 65 percent accuracy, 66 percent weighted F1, and 62 percent macro F1.

The full 40-label dataset remains difficult because several categories have very few samples. For this reason, top-label filtering is used for a reliable 60-80 percent assignment benchmark.

Explainability

Each extracted entity contains its text, label, character position, confidence score, and extraction source.

The explanation layer tells whether an entity came from rules, regex, spaCy, scispaCy, or transformer NER.

This helps users understand why the system produced a result and supports debugging when the extraction is incorrect.

API Design

The FastAPI backend exposes endpoints for report upload, raw text analysis, summary retrieval, entity retrieval, and health checks.

Endpoints include POST /upload-report, POST /analyze, GET /summary, GET /entities, and GET /health.

Swagger documentation is available automatically at http://127.0.0.1:8000/docs when the API is running.

Frontend Dashboard

The Streamlit dashboard allows users to paste report text or upload a PDF.

It displays patient summary, patient details, grouped clinical findings, confidence scores, explainability records, and export options.

Raw structured JSON is kept in the Export tab because it is mainly useful for developers and backend integration.

Privacy And Security

The system avoids logging raw patient text.

Patient identifiers are redacted in structured output.

Uploaded files are cleaned up after processing.

SQLite stores structured analysis results, and production deployments should add authentication, encryption, audit logging, and stricter access control.

Deployment

The project can run locally with Streamlit and FastAPI.

The API can be started using uvicorn app:app --reload.

The dashboard can be started using streamlit run frontend/streamlit_app.py.

Docker support is included through Dockerfile and docker-compose.yml.

Results

The NLP pipeline extracts medical facts such as symptoms, diagnoses, medicines, tests, observations, dates, and patient details.

The classifier reaches the requested 60-80 percent range when evaluated on the top-5 or top-3 frequent specialty benchmark.

The system produces readable reports for users and structured JSON for downstream applications.

Limitations

The system is not a medical device and should not be used as the sole basis for clinical decisions.

NER accuracy depends on installed models and report quality.

OCR quality depends on scan resolution and document formatting.

The Kaggle dataset is imbalanced, so full 40-class classification accuracy is naturally lower.

Future Scope

Fine-tune ClinicalBERT or BioClinicalBERT on domain-specific labeled data.

Add medspaCy section detection and clinical context detection such as negation.

Add RAG-based medical question answering over extracted reports.

Add multilingual OCR and multilingual medical NER.

Add model monitoring, user feedback, and active learning.

Conclusion

The Medical Report Analyzer provides a complete, modular, explainable, and privacy-aware foundation for clinical report NLP.

It demonstrates how practical rule-based extraction, machine learning, transformer-ready NLP, API services, dashboards, and documentation can be combined into one maintainable project.

References

Kaggle Medical Transcriptions Dataset: https://www.kaggle.com/datasets/tboyle10/medicaltranscriptions

spaCy: https://spacy.io

scispaCy: https://allenai.github.io/scispacy/

Hugging Face Transformers: https://huggingface.co/docs/transformers

scikit-learn: https://scikit-learn.org

FastAPI: https://fastapi.tiangolo.com

Streamlit: https://streamlit.io